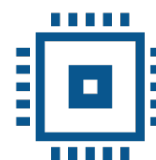




Universitatea *Transilvania* din Braşov
Facultatea de Inginerie Electrică şi Ştiinţa Calculatoarelor
Departamentul Automatică şi Tehnologia Informaţiei



Temă de semestru POO/PCLP3

Jipa Stefan Ciprian
Gr. 4LF442, AIA II

BRAŞOV, 2026

Cuprins

1.	Titlul proiectului.....	2
1.1.	Enunț.....	2
1.2.	Rezolvare proiect.....	2
1.3.	Rezultate.....	3

1. Titlul proiectului

Laborator 7

1.1. Enunț

Se cere extinderea aplicației „Manager de note” cu următoarele funcționalități: Programare orientată pe obiecte Programarea calculatoarelor și limbaje de programare – modulul 3

- Adăugarea unui buton „Sortare” care sortează lista de studenți alfabetic și reafișează datele în ListBox;
- Adăugarea unui buton „Căutare” care, pe baza unui nume introdus într - un TextBox, afișează nota studentului respectiv sau un mesaj dacă studentul nu este găsit;
- Afișarea într - un ListView a studenților cu note peste medie (complementar funcționalității existente pentru studenții sub medie) .

1.2. Rezolvare proiect

lab7.java

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Globalization;
using System.Linq;
using System.Windows.Forms;

namespace lab7
{
    // --- MODELE SI SERVICII (În mod normal stau în foldere separate, dar le-am
    grupat aici) ---

    public class StudentGrade
    {
        public string StudentName { get; }
        public double Grade { get; }

        public StudentGrade(string studentName, double grade)
        {
            StudentName = studentName ?? throw new
            ArgumentException(nameof(studentName));
            Grade = grade;
        }

        public override string ToString() => $"{StudentName} - {Grade:F2}";
    }

    public sealed class GradeBook
    {
        private readonly List<StudentGrade> _entries = new List<StudentGrade>();

        public IReadOnlyList<StudentGrade> Entries => _entries;
        public int Count => _entries.Count;

        public void Add(string studentName, string gradeText)
        {
            var normalizedName = NormalizeStudentName(studentName);
            var grade = ParseGrade(gradeText);
            _entries.Add(new StudentGrade(normalizedName, grade));
        }

        public void RemoveAt(int index)
        {

```

```
        if (index < 0 || index >= _entries.Count)
        {
            throw new ArgumentOutOfRangeException(nameof(index), "Indicele
selectat nu este valid.");
        }
        _entries.RemoveAt(index);
    }

    public double GetAverage()
    {
        EnsureHasEntries();
        return _entries.Average(entry => entry.Grade);
    }

    public StudentGrade GetMaxGradeEntry()
    {
        EnsureHasEntries();
        return _entries.OrderByDescending(entry => entry.Grade).First();
    }

    public StudentGrade GetMinGradeEntry()
    {
        EnsureHasEntries();
        return _entries.OrderBy(entry => entry.Grade).First();
    }

    public IReadOnlyList<StudentGrade> GetStudentsBelowAverage()
    {
        var average = GetAverage();
        return _entries.Where(entry => entry.Grade < average)
            .OrderBy(entry => entry.StudentName)
            .ToList();
    }

    // Exercițiul 1: Sortare alfabetică a listei interne
    public void SortAlphabetically()
    {
        _entries.Sort((a, b) => string.Compare(a.StudentName, b.StudentName,
StringComparison.OrdinalIgnoreCase));
    }

    // Exercițiul 1: Căutare după nume
    public StudentGrade SearchByName(string name)
    {
        var normalizedName = NormalizeStudentName(name);
        return _entries.FirstOrDefault(e => string.Equals(e.StudentName,
normalizedName, StringComparison.OrdinalIgnoreCase));
    }

    // Exercițiul 1: Lista cu studenți peste medie (sau egală)
    public IReadOnlyList<StudentGrade> GetStudentsAboveAverage()
    {
        var average = GetAverage();
        return _entries.Where(entry => entry.Grade >= average)
            .OrderBy(entry => entry.StudentName)
            .ToList();
    }

    private static string NormalizeStudentName(string studentName)
    {
        if (string.IsNullOrEmpty(studentName))
        {
            throw new ArgumentException("Numele studentului este obligatoriu.");
        }
    }
}
```

```
        }
        var normalized = studentName.Trim();
        if (normalized.Any(char.IsDigit))
        {
            throw new ArgumentException("Numele studentului nu poate conține
cifre.");
        }
        return normalized;
    }

    private static double ParseGrade(string gradeText)
    {
        if (string.IsNullOrEmpty(gradeText))
        {
            throw new ArgumentException("Nota studentului este obligatorie.");
        }
        var normalized = gradeText.Trim().Replace(',', '.');
        if (!double.TryParse(normalized, NumberStyles.AllowDecimalPoint,
CultureInfo.InvariantCulture, out var grade))
        {
            throw new ArgumentException("Nota trebuie să fie un număr valid.");
        }
        if (grade < 1 || grade > 10)
        {
            throw new ArgumentOutOfRangeException(nameof(gradeText), "Nota
trebuie să fie în intervalul [1, 10].");
        }
        return grade;
    }

    private void EnsureHasEntries()
    {
        if (_entries.Count == 0)
        {
            throw new InvalidOperationException("Nu există note înregistrate.");
        }
    }
}

// --- FORMULARUL PRINCIPAL ---

public partial class Form1 : Form
{
    private GradeBook _gradeBook = new GradeBook();

    // Controale UI declarate programatic
    TextBox tbNume, tbNota, tbCautare;
    Button btnAdauga, btnSterge, btnCalculeazaMedia, btnAfisareMinMax,
btnStudentiSubMedie;
    Button btnSortare, btnCautare;
    ListBox lbxNote;
    ListView lvStudentiSubMedie, lvStudentiPesteMedie;
    Label lbMedia, lbNrStudenti, lbNotaMax, lbNotaMin;

    public Form1()
    {
        ConstruiesteInterfata();
        ResetStatistics();
        tbNume.Focus();
    }

    private void ConstruiesteInterfata()
    {
```

```
this.Text = "Manager Note Studenți - Lab 7 (Complet)";
this.Size = new Size(950, 650);
this.StartPosition = FormStartPosition.CenterScreen;

int leftCol = 20, rightCol = 140, spacing = 30;

// Panou stânga (Input + Acțiuni baza)
this.Controls.Add(new Label { Text = "Nume student:", Location = new
Point(leftCol, 20), AutoSize = true });
tbNume = new TextBox { Location = new Point(rightCol, 20), Width = 150 };
this.Controls.Add(tbNume);

this.Controls.Add(new Label { Text = "Nota studentului:", Location = new
Point(leftCol, 20 + spacing), AutoSize = true });
tbNota = new TextBox { Location = new Point(rightCol, 20 + spacing),
Width = 150 };
this.Controls.Add(tbNota);

btnAdauga = new Button { Text = "Adaugă", Location = new Point(leftCol,
20 + spacing * 2), Width = 130, BackColor = Color.LightGreen };
btnAdauga.Click += btnAdauga_Click;
this.Controls.Add(btnAdauga);

btnSterge = new Button { Text = "Șterge", Location = new Point(160, 20 +
spacing * 2), Width = 130, BackColor = Color.LightCoral };
btnSterge.Click += btnSterge_Click;
this.Controls.Add(btnSterge);

btnCalculeazaMedia = new Button { Text = "Calculează medie", Location =
new Point(leftCol, 20 + spacing * 3), Width = 270 };
btnCalculeazaMedia.Click += btnCalculeazaMedia_Click;
this.Controls.Add(btnCalculeazaMedia);

btnAfisareMinMax = new Button { Text = "Afișare note min și max",
Location = new Point(leftCol, 20 + spacing * 4), Width = 270 };
btnAfisareMinMax.Click += btnMinMax_Click;
this.Controls.Add(btnAfisareMinMax);

btnStudentiSubMedie = new Button { Text = "Afișare studenți sub/peste
medie", Location = new Point(leftCol, 20 + spacing * 5), Width = 270 };
btnStudentiSubMedie.Click += btnStudentiSubMedie_Click;
this.Controls.Add(btnStudentiSubMedie);

// Labels pentru statistici
lbNrStudenti = new Label { Location = new Point(leftCol, 20 + spacing *
7), AutoSize = true, Font = new Font("Segoe UI", 9, FontStyle.Bold) };
this.Controls.Add(lbNrStudenti);

lbMedia = new Label { Location = new Point(leftCol, 20 + spacing * 8),
AutoSize = true, Font = new Font("Segoe UI", 9, FontStyle.Bold) };
this.Controls.Add(lbMedia);

lbNotaMax = new Label { Location = new Point(leftCol, 20 + spacing * 9),
AutoSize = true, ForeColor = Color.Green };
this.Controls.Add(lbNotaMax);

lbNotaMin = new Label { Location = new Point(leftCol, 20 + spacing * 10),
AutoSize = true, ForeColor = Color.Red };
this.Controls.Add(lbNotaMin);

// ListBox pentru note
```

```
        this.Controls.Add(new Label { Text = "Lista note:", Location = new
Point(320, 20), AutoSize = true });
        lbxNote = new ListBox { Location = new Point(320, 40), Width = 200,
Height = 250 };
        this.Controls.Add(lbxNote);

        // Exercițiul 1: Buton Sortare
        btnSortare = new Button { Text = "Sortare Alfabetică", Location = new
Point(320, 300), Width = 200 };
        btnSortare.Click += btnSortare_Click;
        this.Controls.Add(btnSortare);

        // Exercițiul 1: Secțiune Cautare
        this.Controls.Add(new Label { Text = "Căutare după nume:", Location = new
Point(320, 340), AutoSize = true });
        tbCautare = new TextBox { Location = new Point(320, 360), Width = 120 };
        this.Controls.Add(tbCautare);
        btnCautare = new Button { Text = "Caută", Location = new Point(450, 359),
Width = 70 };
        btnCautare.Click += btnCautare_Click;
        this.Controls.Add(btnCautare);

        // ListView pentru sub medie
        this.Controls.Add(new Label { Text = "Studenti sub medie:", Location =
new Point(550, 20), AutoSize = true });
        lvStudentiSubMedie = new ListView { Location = new Point(550, 40), Width
= 170, Height = 250, View = View.List };
        this.Controls.Add(lvStudentiSubMedie);

        // Exercițiul 1: ListView complementar pentru studenți peste medie
        this.Controls.Add(new Label { Text = "Studenti peste/egal medie:",
Location = new Point(740, 20), AutoSize = true });
        lvStudentiPesteMedie = new ListView { Location = new Point(740, 40),
Width = 170, Height = 250, View = View.List };
        this.Controls.Add(lvStudentiPesteMedie);
    }

    // --- EVENIMENTE SI LOGICA DE BAZA ---

    private void btnAdauga_Click(object sender, EventArgs e)
    {
        try
        {
            _gradeBook.Add(tbNume.Text, tbNota.Text);
            RefreshStudentList();
            ClearInputFields();
            UpdateStatistics();
        }
        catch (Exception ex) when (ex is ArgumentException || ex is
ArgumentOutOfRangeException)
        {
            ShowValidationError(ex.Message);
        }
    }

    private void btnCalculeazaMedia_Click(object sender, EventArgs e)
    {
        try { UpdateStatistics(); }
        catch (InvalidOperationException ex) { ShowValidationError(ex.Message); }
    }

    private void btnMinMax_Click(object sender, EventArgs e)
    {
```

```
        try
        {
            var maxEntry = _gradeBook.GetMaxGradeEntry();
            var minEntry = _gradeBook.GetMinGradeEntry();
            lbNotaMax.Text = $"Nota maximă: {maxEntry.Grade:F2} - {maxEntry.StudentName}";
            lbNotaMin.Text = $"Nota minimă: {minEntry.Grade:F2} - {minEntry.StudentName}";
        }
        catch (InvalidOperationException ex)
        {
            ShowValidationError(ex.Message);
        }
    }

    private void btnStergere_Click(object sender, EventArgs e)
    {
        if (lbxNote.SelectedIndex < 0)
        {
            ShowValidationError("Selectați un student din listă pentru ștergere.");
            return;
        }
        try
        {
            _gradeBook.RemoveAt(lbxNote.SelectedIndex);
            RefreshStudentList();

            if (_gradeBook.Count == 0)
            {
                ResetStatistics();
            }
            else
            {
                UpdateStatistics();
                RefreshStudentsBelowAboveAverage();
            }
        }
        catch (ArgumentOutOfRangeException ex)
        {
            ShowValidationError(ex.Message);
        }
    }

    private void btnStudentiSubMedie_Click(object sender, EventArgs e)
    {
        try
        {
            RefreshStudentsBelowAboveAverage();
        }
        catch (InvalidOperationException ex)
        {
            ShowValidationError(ex.Message);
        }
    }

    // --- EVENIMENTE EXERCITIUL 1 ---

    private void btnSortare_Click(object sender, EventArgs e)
    {
        try
        {
            _gradeBook.SortAlphabetically();
            RefreshStudentList();
        }
    }
}
```

```
        }
        catch (Exception ex)
        {
            ShowValidationError(ex.Message);
        }
    }

    private void btnCautare_Click(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(tbCautare.Text))
        {
            ShowValidationError("Introduceți un nume pentru a căuta.");
            return;
        }

        try
        {
            var result = _gradeBook.SearchByName(tbCautare.Text);
            if (result != null)
            {
                MessageBox.Show($"Student găsit:\nNume: {result.StudentName}\nNota: {result.Grade:F2}",
                                "Rezultat Căutare", MessageBoxButtons.OK,
                                MessageBoxIcon.Information);
            }
            else
            {
                MessageBox.Show("Studentul nu a fost găsit în listă.",
                                "Rezultat Căutare", MessageBoxButtons.OK,
                                MessageBoxIcon.Information);
            }
        }
        catch (Exception ex)
        {
            ShowValidationError(ex.Message);
        }
    }

    // --- FUNCTII AJUTATOARE ---

    private void RefreshStudentList()
    {
        lbxNote.BeginUpdate();
        lbxNote.Items.Clear();
        foreach (var entry in _gradeBook.Entries)
        {
            lbxNote.Items.Add(entry);
        }
        lbxNote.EndUpdate();
    }

    private void RefreshStudentsBelowAboveAverage()
    {
        lvStudentiSubMedie.BeginUpdate();
        lvStudentiPesteMedie.BeginUpdate();

        lvStudentiSubMedie.Items.Clear();
        lvStudentiPesteMedie.Items.Clear();

        // Parcurgem pentru afișare studenți cu nota < medie
        foreach (var entry in _gradeBook.GetStudentsBelowAverage())
        {
            lvStudentiSubMedie.Items.Add(entry.ToString());
        }
    }
}
```

```
    }

    // Exercițiul 1: Parcurgem pentru afișare studenți cu nota >= medie
    foreach (var entry in _gradeBook.GetStudentsAboveAverage())
    {
        lvStudentiPesteMedie.Items.Add(entry.ToString());
    }

    lvStudentiSubMedie.EndUpdate();
    lvStudentiPesteMedie.EndUpdate();
}

private void UpdateStatistics()
{
    if (_gradeBook.Count == 0) return;
    var average = _gradeBook.GetAverage();
    lbNrStudenti.Text = $"Număr total studenți: {_gradeBook.Count}";
    lbMedia.Text = $"Media notelor: {average:F2}";
}

private void ResetStatistics()
{
    lbNrStudenti.Text = "Număr total studenți: 0";
    lbMedia.Text = "Media notelor: -";
    lbNotaMax.Text = "Nota maximă: -";
    lbNotaMin.Text = "Nota minimă: -";
    lvStudentiSubMedie.Items.Clear();
    lvStudentiPesteMedie.Items.Clear();
}

private void ClearInputFields()
{
    tbNume.Clear();
    tbNota.Clear();
    tbNume.Focus();
}

private static void ShowValidationError(string message)
{
    MessageBox.Show(message, "Date invalide", MessageBoxButtons.OK,
    MessageBoxIcon.Warning);
}
}
```

1.3. Rezultate

Manager Note Studenți - Lab 7 (Complet)

Nume student:
Nota studentului:

Număr total studenți: 3
Media notelor: 7.00
Nota maximă: 8.00 - Marian Popa
Nota minimă: 5.00 - Jipa Jipa

Lista note:
Marian Popa - 8.00
Jipa Jipa - 5.00
Alexei - 8.00

Căutare după nume:

Studenți sub medie:
Jipa Jipa - 5.00

Studenți peste/egal medie:
Alexei - 8.00
Marian Popa - 8.00